

Kubernetes & Flannel Troubleshooting

Production-Style Scenario — DevOps & SRE Interview Preparation Guide

★ 1-MINUTE ELEVATOR PITCH

"I troubleshooted a production Kubernetes cluster issue where a worker node showed a Flannel CNI failure with the kube-flannel pod in CrashLoopBackOff. Checking previous container logs revealed Failed to check br_netfilter due to a missing /proc/sys/net/bridge/bridge-nf-call-iptables. I diagnosed that the br_netfilter kernel module was unloaded, loaded it via modprobe, set the required sysctl parameters (bridge-nf-call-iptables, bridge-nf-call-ip6tables, ip_forward), and persisted them in /etc/modules-load.d/ and /etc/sysctl.d/. After restarting the pod from the control plane, the CNI and node recovered to Ready. Later, encountering a connection refused: 6443 error from the worker, I verified API server process state, port binding, and kubeconfig alignment, systematically isolating Linux kernel networking from Kubernetes control plane access."

Part 1: Detailed Interview Scenario Q&A

1. What issue did you face in your Kubernetes cluster?

I had a Kubernetes cluster with one master and worker nodes. One worker node showed networking issues, and its kube-flannel DaemonSet pod was stuck in CrashLoopBackOff.

I inspected the pod status and examined the previous failed container logs:

```
kubectl get pods -n kube-flannel -o wide
kubectl logs -n kube-flannel <flannel-pod-name> --previous
```

```
Failed to check br_netfilter: stat /proc/sys/net/bridge/bridge-nf-call-iptables: no such file or directory
```

2. What did this error mean and how did you resolve it?

The Linux kernel module br_netfilter was missing/unloaded. Flannel and Kubernetes rely on bridge traffic passing through iptables for packet filtering and routing.

```
lsmod | grep br_netfilter      # Returned empty
modprobe br_netfilter          # Loaded the kernel module
lsmod | grep br_netfilter      # Verified active module
```

3. What sysctl parameters were configured?

Enabled bridge packet filtering and IPv4 packet forwarding:

```
sysctl -w net.bridge.bridge-nf-call-iptables=1
sysctl -w net.bridge.bridge-nf-call-ip6tables=1
sysctl -w net.ipv4.ip_forward=1
```

Verified current values:

```
sysctl net.bridge.bridge-nf-call-iptables
sysctl net.bridge.bridge-nf-call-ip6tables
sysctl net.ipv4.ip_forward
```

4. How did you make this configuration permanent across reboots?

Created module load and sysctl configuration files:

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system
```

5. What recovery steps did you perform after fixing br_netfilter?

Restarted the Flannel pod from the control plane node:

```
kubectl delete pod -n kube-flannel <flannel-pod-name>
```

Kubernetes automatically recreated the pod via DaemonSet. Verification confirmed recovery:

```
kubectl get pods -n kube-flannel -o wide --> 1/1 Running
kubectl get nodes --> kubworker-node2 Ready
```

6. What was the second issue with "connection refused on port 6443"?

Executing kubectl from the worker returned:

```
Unable to connect to the server: dial tcp 172.31.12.72:6443: connect: connection refused
```

This indicated an inability to establish a TCP handshake on port 6443. I checked whether kubeconfig pointed to the master API Server or mistakenly to the worker node IP:

```
kubectl config view --minify | grep server
grep server: /etc/kubernetes/kubelet.conf
```

7. How did you verify API Server status and connectivity?

On Master: Verified container runtime and listening port:

```
sudo crictl ps | grep kube-apiserver
sudo ss -lntp | grep 6443
```

On Worker: Tested network layer reachability to master:

```
nc -zv <MASTER-IP> 6443
```

Part 2: Core Architectural Concepts

Flannel & VXLAN Architecture

Flannel is a lightweight Container Network Interface (CNI) provider that allocates a subnet lease to each node from a larger pre-configured cluster IP range. Using the **VXLAN** backend, it encapsulates Layer 2 Ethernet frames into Layer 4 UDP packets (port 4789/8472), allowing pod-to-pod communication across different physical nodes without requiring specialized router BGP configurations.

br_netfilter & IP Forwarding

`br_netfilter` enables packets traversing Linux bridge interfaces to be inspected and modified by iptables / netfilter chains (PREROUTING, FORWARD, POSTROUTING). Without it, bridged traffic bypasses iptables, breaking Kubernetes Service ClusterIP proxying, Pod CIDR NAT, and NetworkPolicies. `net.ipv4.ip_forward` allows the Linux kernel to route packets between internal veth pairs and physical NIC interfaces.

Part 3: Systematic 6-Step Node Troubleshooting Matrix

Layer	Diagnostic Command	Target Inspection Items
1. Node Level	<code>kubectl get nodes -o wide</code> <code>kubectl describe node <name></code>	Node status (Ready / NotReady), Conditions (MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable).
2. Kubelet	<code>systemctl status kubelet</code> <code>journalctl -u kubelet -xe --no-pager</code>	Node registration status, TLS bootstrap issues, runtime endpoint connection failures.
3. Container Runtime	<code>systemctl status containerd</code> <code>sudo crictl ps / crictl pods</code>	CRI socket responsiveness, runtime daemon state, hung sandbox containers.
4. CNI Network	<code>kubectl get pods -n kube-flannel -o wide</code> <code>kubectl logs -n kube-flannel <pod></code>	CNI DaemonSet health, subnet lease errors, iptables/vxlan interface config.
5. Kernel & Routing	<code>ip addr & ip route</code> <code>sysctl net.ipv4.ip_forward</code>	Flannel interface (flannel.1), routing table entries, kernel packet forwarding.
6. Control Plane	<code>ss -lntp grep 6443</code> <code>nc -zv <MASTER-IP> 6443</code>	API Server port 6443 listening status, security groups, firewall rules.

 **Interview Delivery Tip:** Never say *"I fixed it by deleting the pod."* Instead, state: **"I identified the root cause in the Linux kernel networking subsystem, applied and persisted the missing kernel parameters, and then recycled the CNI pod to allow the worker node to transition back to Ready."**